

1. Introduction

This report documents my implementation of the cross-season temporal image matching assessment. The objective was to match Summer/Spring reference images against Winter/Snow target images from the CMU-Seasons evaluation subset, estimate a geometric alignment between each pair, generate match and warp visualisations, and profile the runtime on GPU.

The final system uses XFeat* semi-dense deep feature matching, followed by USAC_MAGSAC homography estimation. I selected this approach because the problem has strong seasonal appearance shifts: snow changes the ground texture, leaf cover changes the tree canopy, and lighting/shadow conditions vary across images. A learned local feature matcher was therefore a better starting point than relying only on handcrafted descriptors.

The final run achieved a mean inlier ratio of 0.883 across five image pairs and a mean end-to-end latency of 48.14 ms per pair on CUDA. The repository contains all scripts, output images, reports, ONNX assets, and a README with setup and optimisation details.

2. Dataset and Submission Structure

The task resources pointed to a CMU-Seasons target evaluation subset. In the submitted repository I kept the data loading structure close to the requested format:

- data/summer/ contains the Summer/Spring reference images.
- data/winter/ contains the Winter/Snow target images.
- data/pairs.csv maps each source image to the corresponding target image.

The required output folder is included as output/. It contains five match visualisation images in output/matches/, five homography warp alignment images in output/warps/, and latency/validation reports in output/reports/.

3. Methodology

3.1 Pipeline Overview

For each input pair, the implementation follows this sequence: image loading and resizing, feature extraction, cross-season matching, robust homography estimation, match visualisation, image warping, and latency logging. The main script is src/run_matching.py.

- Images are resized using max_side=512 to improve runtime while retaining enough structural detail for reliable matching.
- XFeat* produces semi-dense feature correspondences between the Summer/Spring and Winter/Snow images.
- USAC_MAGSAC estimates the homography and rejects geometrically inconsistent matches.
- The Summer/Spring image is warped into the Winter/Snow frame using cv2.warpPerspective.
- Latency is logged for data loading, feature extraction/matching, and homography estimation.

3.2 Reasoning Behind Technical Choices

I chose a deep feature matching pipeline because cross-season matching is a difficult setting for purely classical local descriptors. Snow can remove ground texture. Foliage changes the appearance of trees, and seasonal lighting changes can make gradient-based features unstable. A learned local feature matcher is more likely to retain correspondences on stable structures.

I used USAC_MAGSAC rather than a direct fit because the tentative matches contain outliers. In this task, outliers are not only random matching mistakes; they can also come from valid-looking seasonal regions such as snow patches, shadows and etc. Robust estimation is therefore important before the homography is trusted.

I tuned the final resolution after seeing the runtime-quality trade-off. With max_side=640, the cleaned repository run was close to or above the 50 ms latency limit. Reducing to max_side=512 preserved a strong inlier ratio while keeping the mean latency below the target.

4. Mathematical Formulation

The homography is a 3×3 projective transformation that maps a source image point to the target image frame. I used it as an approximate alignment model between the Summer/Spring and Winter/Snow images.

$$x_S = [x, y, 1]^T, \quad x_W = [u, v, 1]^T$$

$$\lambda x_W = H x_S, \quad H \in \mathbb{R}^{3 \times 3}$$

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

After multiplying by the homography, the projected target coordinates are obtained by normalising with the third homogeneous coordinate:

$$\hat{u} = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad \hat{v} = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

For a manually selected or matched correspondence, the reprojection error measures how far the projected source point is from its expected target location:

$$e_i = \sqrt{(u_i - \hat{u}_i)^2 + (v_i - \hat{v}_i)^2}$$

The RANSAC inlier ratio is used as the main matching robustness metric:

$$\text{Inlier Ratio} = \frac{N_{\text{inliers}}}{N_{\text{matches}}}$$

The runtime metric is measured as the end-to-end latency per image pair:

$$T_{\text{total}} = T_{\text{load}} + T_{\text{feature/match}} + T_{\text{homography}}$$

5. Visual Evidence

The repository includes all five match visualisations and all five warp visualisations. The match images show side-by-side images with accepted feature correspondences, while the warp images show the Summer/Spring image transformed into the Winter/Snow image frame.



6. Quantitative Results and Performance Analysis

6.1 Matching Robustness

All five image pairs passed the 60% inlier-ratio target after robust filtering. The mean inlier ratio was 0.883, which indicates that most accepted correspondences were geometrically consistent with the estimated homography.

Pair	Matches	Inliers	Inlier ratio	Total latency
pair_001	359	340	0.947	48.97 ms
pair_002	285	264	0.926	48.46 ms
pair_003	248	213	0.859	47.11 ms
pair_004	221	173	0.783	48.18 ms
pair_005	362	326	0.901	48.01 ms

6.2 Runtime Efficiency

The final measured mean latency was 48.14 ms per pair, which is below the 50 ms target. The feature extraction/matching stage dominates the runtime, while homography estimation is relatively small in comparison.

Stage	Mean time
Data loading	14.29 ms
Feature extraction / matching	32.84 ms
Homography matrix estimation	1.01 ms
Total end-to-end	48.14 ms

The latency is close to the target, so I reported the latest measured number rather than the fastest earlier run. This is a more conservative and reproducible result after restructuring the repository to the requested data/summer and data/winter format.

6.3 ONNX / Deployment Benchmark

The XFeat backbone was exported to ONNX and benchmarked using ONNX Runtime. The final deployment path uses ONNX Runtime CUDAExecutionProvider. TensorRTEExecutionProvider was tested, but the local system did not include the required TensorRT runtime library libnvinfer.so.10, so I did not claim TensorRT results.

Item	Value
Exported model	model/xfeat_480x640.onnx
Input shape	1 x 3 x 480 x 640
ONNX Runtime CPU	12.64 ms
ONNX Runtime CUDA	2.70 ms
CUDA speedup	4.67x
TensorRT status	Tested, but runtime library libnvinfer.so.10 was unavailable

7. Task 2: Alignment Accuracy Discussion

For Task 2, I implemented homography estimation and produced warped alignment images for all five pairs. The warp is computed with `cv2.warpPerspective(source_image, H, target_image_size)`.

The rubric asks for a four-corner warp check against ground-truth target corners. In the CMU-Seasons subset used for this submission, explicit pairwise ground-truth homography matrices or corner annotations were not available for the curated pairs. I therefore avoided reporting fabricated ground-truth corner errors.

As an alternative quantitative check, I used manually selected static landmarks where reliable landmarks were visible. I chose roof corners, curb corners, pole bases, building edges, and other stable structures. The robust median landmark reprojection error was below 5 pixels for the validated landmark pairs.

Validation item	Result
Validated landmark pairs	4

Mean manual reprojection error	3.35 px
Average median landmark reprojection error	2.92 px
Robust median alignment check	< 5 px on validated pairs
Excluded from manual landmark validation	pair_005 only

pair_005 is included in matching, warping, and latency evaluation. It is excluded only from manual landmark validation because it contains no reliable static man-made control points and is dominated by trees, snow, and seasonal foreground changes.

8. Personal Findings and Discovery Notes

This was the most useful part of the task for me because the first working result was not automatically the best result. I had to look at the match images, the inlier ratios, and the warped outputs together before deciding whether a pair was actually reliable.

8.1 What I Noticed While Testing

- Some pairs that looked visually similar to me did not always give strong geometric consistency after RANSAC.
- The ground region was often weak for matching because snow changes or removes grass, road texture, shadows, lane details, and curb contrast.
- Accepted matches often appeared in the upper half of the image because tree trunks, rooflines, poles, and far-field structures were more repeatable across seasons.
- Resolution had a clear speed-quality trade-off. The 512 setting gave a good balance between keeping structural details and meeting the runtime target.
- The warp was strongest on stable structures and less reliable on seasonal foreground objects, which is expected when a single homography is used on a non-planar outdoor scene.

8.2 Match Distribution Observation

In several visualisations, many accepted matches lie around the upper half of the image. At first this looked suspicious, but it became explainable after inspecting the scenes. The road and ground plane are exactly the regions most affected by seasonal change: grass becomes snow, shadows shift, road texture becomes weaker, and small ground details disappear.

The robust estimator therefore keeps matches mostly on structures that remain stable across seasons: tree trunks, rooflines, poles, curbs, building edges, and far-field boundaries. For a localisation pipeline, this is actually reasonable because stable landmarks are more useful than seasonal foreground texture.

8.3 Implementation Experience

A practical issue I faced was deployment. ONNX export was successful, and ONNX Runtime CUDA gave a measurable speedup. TensorRT could not be reported because the required runtime library was missing on my system. I chose to document this directly instead of hiding the failed test. This made the final submission more honest and reproducible.

The project also made me appreciate that a robotics perception pipeline is not only model inference. It needs data organisation, geometric verification, runtime profiling, error handling for weak pairs, reproducible outputs, and clear documentation of assumptions.

9. Limitations

The main limitation is that a single homography is only an approximation for outdoor scenes. A street scene contains roads, buildings, trees, vehicles, and other objects at different depths. A homography cannot align all

depths perfectly, so the warp should be interpreted as an alignment check on dominant stable structures rather than a full 3D registration.

Another limitation is the lack of official pairwise ground-truth homographies for the selected pairs. This affects four-corner ground-truth error values. My workaround was to report landmark reprojection validation honestly and provide the warp outputs for visual inspection.

10. Future Improvements

- Use explicit ground-truth corner annotations or pose metadata if provided, so that the four-corner error can be computed directly.
- Compare XFeat* against a classical SIFT baseline and a SuperPoint/SuperGlue or LightGlue configuration.
- Use depth or camera pose information to move beyond a single homography for non-planar outdoor scenes.
- Try TensorRT or OpenVINO on a properly configured machine to complete the deployment comparison.
- Add a small ablation table over max_side values to make the speed-quality trade-off more formal.

11. Repository Checklist

Requirement	Where it is provided
Python scripts	src/ and tools/
Model/export assets	model/xfeat_480x640.onnx and model/*.json
5 match visualisations	output/matches/
5 warped alignment images	output/warps/
Latency profiling table	README.md and output/reports/latency.csv
Setup instructions	README.md
Optimisation breakdown	README.md and Section 6.3 of this report

12. Conclusion

The final submission implements the requested cross-season matching pipeline with deep feature matching, robust homography estimation, visual outputs, latency profiling, and ONNX Runtime CUDA benchmarking. The results pass the inlier-ratio target and the runtime target.

The main unresolved point is the exact four-corner ground-truth error, because explicit ground-truth homographies were not available for the selected pairs. I documented this limitation and used manual landmark reprojection validation as an honest substitute.